

PINAKA

APPLICATION v1

A simple and intuitive guide to how PINAKA starts, runs, and shuts down

Goal of this document: not just to memorize the code, but to make the startup flow feel natural.

The single most important sentence:

PINAKA Engine provides the common way an application runs; the client decides which specific application PINAKA should run.

1. First: What are Engine, Studio, and Game?

Before understanding Application, separate these three ideas. This is where most of the confusion came from.

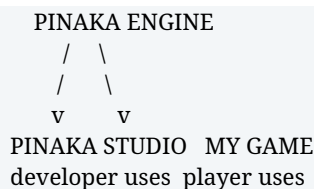
Thing	Easy meaning	In PINAKA
Engine	The machinery	PINAKA Engine
Studio	The program you use to make a game	PinakaStudio.exe
Game	The final program a player runs	A future MyGame.exe

A simple analogy:

Microsoft Word -> used to make documents
 Photoshop -> used to make images
 Pinaka Studio -> used to make games

PINAKA Engine -> the machinery Studio uses underneath

So Studio is not the game. Studio is the tool you, the developer, open while building the game.



Mental shortcut: when you see Studio, think: 'the program I open to build a game with PINAKA.'

2. Why Do We Need pke::Application?

Technically, we could put everything directly inside main(). For a tiny program, this would work:

```

int main()
{
    initEngine();

    while (running)
    {
        update();
    }

    shutdown();
    return 0;
}
  
```

But if several programs use PINAKA, every program would have to repeat the same startup, loop, and shutdown logic.

Studio main() -> init -> loop -> shutdown
 Game main() -> init -> loop -> shutdown
 Sandbox main() -> init -> loop -> shutdown

Instead, PINAKA Engine provides one reusable Application class containing that common lifecycle.

```
pke::Application
|
+-- initialize
+-- run the main loop
+-- allow update customization
+-- shut down
```

Application is the thing that manages the lifetime of a PINAKA-powered program.

3. Why Does StudioApplication Inherit from pke::Application?

Because Studio is one specific kind of PINAKA application.

```
class StudioApplication : public pke::Application
{
public:
    void update() override;
};
```

In normal English, this means:

StudioApplication IS A PINAKA Application, but with Studio-specific behavior.

The base Application knows how every PINAKA program should run. Studio only needs to provide the parts that are special to Studio.

```
pke::Application
|
| owns the common lifecycle
v
run()
-> initialize
-> while running
    update()
-> shutdown

StudioApplication
|
| customizes
v
update()
```

This prevents Studio from rewriting the entire engine lifecycle.

4. The Application v1 Class

```
namespace pke
{
    class PK_API Application
    {
    public:
        Application() : m_Running(false) {}
        virtual ~Application() = default;

        void run();

    protected:
        virtual void update();
        void requestShutdown();
    };
}
```

```

private:
    void initEngine();
    void shutdown();

private:
    bool m_Running;
};

Application* CreateApplication();
}

```

What each member means

Member	Easy meaning	Why it is there
m_Running	The ON/OFF switch for the loop	true = keep running; false = stop
run()	Run this application	Owns the complete lifecycle
update()	Do one frame of client-specific work	Studio can override it
requestShutdown()	Please stop running	Controlled way to end the loop
initEngine()	Prepare the application/engine	Internal lifecycle step
shutdown()	Clean up before exit	Internal lifecycle step

5. The Main Loop - The Heart of the Application

```

void Application::run()
{
    initEngine();

    while (m_Running)
    {
        update();
    }

    shutdown();
}

```

Read this code like English:

1. Prepare the application.
2. As long as the application is allowed to run, perform one update.
3. When running becomes false, leave the loop.
4. Clean up and return.

```

m_Running = true

update
update
update
update
...
someone requests shutdown
...
m_Running = false

```

```
loop ends
shutdown
```

The loop is not 'keeping the engine alive' by magic. It simply keeps repeating work until the running switch becomes false.

6. Why Is update() Virtual?

Application owns the loop, but PINAKA does not know what every future client wants to do during an update.

```
while (m_Running)
{
    update();
}
```

If the actual object is StudioApplication, C++ uses virtual dispatch and calls StudioApplication::update().

```
Application::run()
|
v
update()
|
| virtual function
v
StudioApplication::update()
```

So the engine controls WHEN update happens, while Studio controls WHAT its update does.

This is runtime polymorphism with a practical purpose, not just an interview example.

7. Why requestShutdown() Instead of Directly Changing m_Running?

m_Running is private because Application owns its own lifecycle state.

```
// Bad idea for derived/client code:
m_Running = false; // direct internal access
```

Instead, Studio asks Application to stop:

```
requestShutdown();
```

Then Application changes its own internal state.

```
void Application::requestShutdown()
{
    m_Running = false;
}
```

Think of a TV:

You do not open the TV and manually change an internal circuit.
You press the Power button.

Studio does not directly change m_Running.
Studio calls requestShutdown().

Encapsulation in plain English: the object that owns the state controls how that state changes.

8. Why Do We Need CreateApplication()?

This is the extra layer we added after the basic Application flow was understood.

PINAKA Engine wants to own the common startup procedure, but the engine must not know about StudioApplication.

Engine should know:

pke::Application

Engine should NOT know:

StudioApplication

So the engine declares a question:

```
namespace pke
{
    Application* CreateApplication();
}
```

The current client provides the answer. Pinaka Studio implements it:

```
pke::Application* pke::CreateApplication()
{
    return new StudioApplication();
}
```

In simple words:

Engine asks: 'Which Application should I run?' Studio answers: 'Run StudioApplication.'

It does not literally search all client applications at runtime. Each executable that uses this startup pattern provides the definition it needs.

9. Why Does CreateApplication Return Application*?

Because StudioApplication inherits from Application. A base-class pointer can point to a derived object.

```
pke::Application* app = new StudioApplication();
```

This is useful because the engine can work only with the generic Application type and remain completely unaware of the Studio class.

Engine sees:

Application* app

Actual object:

StudioApplication

That separation is one of the key reasons the engine remains reusable.

10. EntryPoint.h - Who Owns main() Now?

Originally Studio had its own main():

```
int main()
{
    StudioApplication app;
    app.run();
}
```

In Application v1, PINAKA provides a reusable entry-point implementation:

```
int main()
{
    pke::Application* app = pke::CreateApplication();

    app->run();

    delete app;

    return 0;
}
```

The important subtle point is this:

EntryPoint.h physically lives inside PinakaEngine, but main() is compiled into PinakaStudio.exe because Studio includes that header.

```
PinakaEngine/Core/EntryPoint.h
|
| included by
v
PinakaStudio source file
|
v
main() becomes part of PinakaStudio.exe
```

So the DLL itself is not the executable starting point. Studio is still the executable being launched.

11. Complete Startup Flow - From Double Click to Exit

```
1. User opens PinakaStudio.exe
|
v
2. main() runs
   (code supplied by PINAKA EntryPoint.h)
|
v
3. main() calls pke::CreateApplication()
|
v
4. Studio's implementation returns:
   new StudioApplication()
|
v
5. Engine stores it as:
   pke::Application* app
|
v
6. app->run()
|
v
7. Application::initEngine()
|
v
8. m_Running = true
|
v
9. while (m_Running)
   -> virtual update()
   -> StudioApplication::update()
|
```

```

      v
10. Eventually requestShutdown()
      |
      v
11. m_Running = false
      |
      v
12. loop ends
      |
      v
13. shutdown()
      |
      v
14. run() returns
      |
      v
15. delete app
      |
      v
16. main() returns
      |
      v
17. PinakaStudio.exe exits

```

12. Why Is the Destructor Virtual?

EntryPoint stores the object using a base pointer:

```
pke::Application* app = pke::CreateApplication();
```

But the real object may be StudioApplication. Later we do:

```
delete app;
```

A virtual base destructor ensures C++ destroys the complete derived object correctly.

```

delete Application*
      |
      v
StudioApplication destructor
      |
      v
Application destructor

```

Rule to remember: if a class is meant to be used polymorphically as a base class, its destructor should normally be virtual.

13. Why PK_API on Application but Not StudioApplication?

pke::Application belongs to PinakaEngine.dll and is used by PinakaStudio.exe, so it crosses the DLL boundary.

```

PinakaEngine.dll
  |
  | exports/imports
  v
pke::Application
  |
  v
PinakaStudio.exe

```


Therefore Application uses PK_API.

```
class PK_API Application
```

StudioApplication lives inside the Studio executable. It is not an engine class being exported from PinakaEngine.dll, so it does not need the engine export macro.

```
class StudioApplication : public pke::Application
```

14. Why Tests Has Its Own main()

EntryPoint.h is special because it contains main(). It must not be included in a common header that every project receives.

```
PinakaStudio.exe
-> uses PINAKA EntryPoint
-> uses CreateApplication()

Tests.exe
-> has its own test main()
-> does NOT use PINAKA EntryPoint
```

This is why putting EntryPoint.h inside a general include such as PinakaEngineInc.h caused linker/startup confusion.

Application.h is reusable API. EntryPoint.h is executable startup code. Treat them differently.

15. Current Studio Test

```
void StudioApplication::update()
{
    std::cout << "Update Studio Application" << std::endl;
    requestShutdown();
}
```

Right now requestShutdown() is called immediately only so we can prove the architecture works without an actual window.

Later, the flow will become:

```
User clicks window X
|
v
Window system receives close request
|
v
Application is told to close
|
v
requestShutdown()
|
v
m_Running = false
|
v
main loop ends
```

16. The Most Intuitive Mental Model

Imagine a manager and a worker:

Application = Manager

Manager says:

- prepare everything
- keep working while we are open
- every turn, call update()
- stop when shutdown is requested
- clean up

StudioApplication = Specific worker

Studio says:

- when you call update(), here is the Studio-specific work I want to do

CreateApplication is simply the point where the client tells the manager system which specific worker/application should exist.

Application controls the lifecycle. Studio customizes the application. EntryPoint starts it. CreateApplication connects the engine to the specific client.

17. What Application v1 Does NOT Do Yet

- It does not create a real operating-system window yet.
- It does not receive window close or resize events yet.
- It does not process keyboard or mouse input yet.
- It does not calculate delta time yet.
- It does not render anything yet.
- It does not own a scene or game objects yet.

That is intentional. We built only enough architecture to understand and prove the application lifecycle.

Application v1 is not 'finished forever'. It is a clean first version that future subsystems will connect to.

18. What Comes Next

The next natural problem is: how does the application know the user clicked the window close button?

```

Application
|
v
Window <-- next
|
v
Window Close Event
|
v
requestShutdown()
```

That leads us to the Window abstraction, and after that to the Event system.

19. Interview Quick Revision

Use this section when you want to revise Application v1 in a few minutes.

30-second explanation

pke::Application is PINAKA's reusable base class for the common application lifecycle. It owns initialization, the main loop, running state, and shutdown. StudioApplication derives from it and overrides update() for Studio-specific work. CreateApplication() lets the client provide its concrete Application without the engine knowing about StudioApplication, and EntryPoint.h provides the reusable main() startup flow.

Key C++ concepts

- Inheritance: StudioApplication is an Application.
- Virtual functions: update() dispatches to StudioApplication::update().
- Runtime polymorphism: Application* can point to a StudioApplication object.
- Encapsulation: m_Running stays private; shutdown is requested through a method.
- Virtual destructor: deleting through Application* safely destroys the derived object.
- DLL boundary: PK_API is on reusable engine classes that cross from PinakaEngine.dll to the client.

Likely interview questions

Why not keep the loop in main()? Because all PINAKA clients would duplicate the same lifecycle. The base Application centralizes it.

Why inherit StudioApplication from Application? Studio is one specific PINAKA application and should reuse the common lifecycle while customizing only its own behavior.

Why is update() virtual? The base class owns when updates happen, while the derived client decides what its update does.

Why is m_Running private? Application owns its lifecycle state. Derived code should request a transition, not directly modify internals.

Why CreateApplication()? It lets the engine obtain a concrete client Application without depending on StudioApplication.

Why return Application*? A base pointer can refer to a derived StudioApplication, allowing the engine to work only with the generic interface/base.

Why virtual destructor? Because EntryPoint deletes the object through an Application pointer.

Why does EntryPoint.h live in Engine but main() end up in Studio? Because Studio includes the header, so the main function is compiled into the Studio executable.

One-line memory hook

Engine controls HOW the app runs. Client decides WHICH app runs. Derived app decides WHAT its custom update does.

Final flow to remember

```
PinakaStudio.exe
|
v
EntryPoint main()
```

```
|  
v  
CreateApplication()  
|  
v  
new StudioApplication()  
|  
v  
Application::run()  
|  
+-- init  
+-- loop -> virtual update()  
+-- shutdown  
|  
v  
delete app  
|  
v  
exit
```

APPLICATION v1 - COMPLETE